ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN – TIN





BÁO CÁO
KIẾN TRÚC MÁY TÍNH

Chủ đề: Thiết kế SoC đồng hồ điện tử WatchChip trên CPU MIPS tự thiết kế (đơn chu kỳ và pipeline)

Sinh viên: **Đoàn Danh Long** *MSSV*
TV1 — chưa điền *MSSV*
TV2 — chưa điền *MSSV*
TV3 — chưa điền *MSSV*
TV4 — chưa điền *MSSV*
TV5 — chưa điền *MSSV*
Lớp: **chưa điền**
Giảng viên giảng dạy: **chưa điền**

HÀ NỘI, 12/2026

MỤC LỤC

LỜI GIỚI THIỆU **v**

1 GIỚI THIỆU **1**

1.1 Bài toán . . . 1

1.2 Cơ sở lý thuyết: kiến trúc tập lệnh MIPS . . . 1

1.2.1 Tổng quan MIPS ISA . . . 1

1.2.2 Tập thanh ghi . . . 2

1.2.3 Định dạng lệnh và 28 lệnh được hỗ trợ . . . 2

1.3 Cơ sở thiết kế . . . 4

1.3.1 CPU đơn chu kỳ . . . 4

1.3.2 Kỹ thuật đường ống (pipelining) . . . 4

1.3.3 Các loại xung đột và kỹ thuật xử lý . . . 4

1.3.4 Ngắt và CP0 . . . 4

2 PHƯƠNG PHÁP LUẬN VÀ THIẾT KẾ **6**

2.1 Công cụ và cấu trúc mã nguồn . . . 6

2.1.1 Phạm vi của đề tài . . . 6

2.1.2 Công cụ . . . 6

2.1.3 Cây thư mục . . . 6

2.1.4 Cách chạy testbench . . . 7

2.2 Thông số và cổng của CPU . . . 8

2.3 CPU G1: đơn chu kỳ . . . 8

2.3.1 Datapath . . . 8

2.3.2 Một chu kỳ CPU gồm hai pha . . . 10

2.3.3 Đường đi của `lw $t0, -252($zero)` . . . 11

2.3.4 Vì sao chọn: G1 hai pha . . . 11

2.4 Khối điều khiển và bảng chân lý . . . 12

2.4.1 Vì sao chọn: giải mã một cấp . . . 14

2.5 CPU G2: pipeline 5 tầng . . . 14

2.6 Xử lý xung đột . . . 15

2.7 ALU, nhân, chia và lệnh `bcd` . . . 15

2.8 CP0 và ngắt . . . 15

2.9 Đoạn code chính . . . 16

2.10 Tổng thể SoC 16
2.11 Bộ điều khiển hiển thị 16
2.12 Timebase, hiệu chỉnh CAL và các bộ định thời 16
2.13 Firmware 17

3 KIỂM THỬ VÀ KẾT QUẢ **18**
3.1 Chiến lược kiểm thử 18
3.2 Trace và dạng sóng 18
3.3 Hiệu năng G1 và G2: CPI tách theo nguyên nhân 18
3.4 Khảo sát thiết kế (DSE) và tài nguyên 18
3.5 Thí nghiệm E1′–E4′ 18
3.5.1 E1′, E2′: hiệu chỉnh CAL 18
3.5.2 E3: polling, ngắt và capture 19
3.5.3 E4′: bóng ma khi quét 7 đoạn 19
3.6 Tổng kết: đạt và chưa đạt 20

4 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **21**
4.1 Kết quả đạt được 21
4.2 Hạn chế 21
4.3 Hướng phát triển 21

A PHÂN CÔNG CÔNG VIỆC **22**

B KẾ HOẠCH VÀ TIẾN ĐỘ **23**

C HƯỚNG DẪN CHẠY LẠI **24**

D TRÍCH TRACE VÀ LOG **25**

TÀI LIỆU THAM KHẢO **26**

DANH MỤC HÌNH VẼ

Hình 2.1 Datapath CPU G1. Nền xanh: khối cập nhật ở pha A; nền cam: khối làm việc ở pha B; trắng: mạch tổ hợp; nét đứt: tín hiệu điều khiển từ `control`. Mọi nhãn là tên tín hiệu hoặc module trong `src/cpu_g1.v`. 9
Hình 2.2 Datapath CPU G2 đầy đủ (H3) 15
Hình 2.3 Khối forwarding và hazard unit (H4) 15
Hình 2.4 Sơ đồ vào và ra ngắt (H5) 16
Hình 2.5 Sơ đồ tổng thể SoC WatchChip (H1) 16

Hình 3.1 Dạng sóng: load-use bubble, flush nhánh, `busy` freeze, vào/ra ngắt (H8) . 18
Hình 3.2 Chi phí đổi ms sang chữ số (H9) 19
Hình 3.3 Bóng ma theo thời gian tắt của chân đoạn, ở 3 giá trị `G_SLOT` 19

DANH MỤC BẢNG BIỂU

Bảng 1.1 Các khuôn lệnh của WatchChip. 2
Bảng 1.2 28 lệnh của WatchChip: mã hóa và ý nghĩa. 2

Bảng 2.1 Công cụ dùng trong đề tài. 7
Bảng 2.2 Tham số, cổng và bộ nhớ của CPU (giống nhau ở G1 và G2). 8
Bảng 2.3 Việc của từng pha trong một chu kỳ CPU của G1. 10
Bảng 2.4 Đường đi của một lệnh `lw` đọc ngoại vi trong G1. 11
Bảng 2.5 Ba cách tổ chức bộ nhớ cho G1. 12
Bảng 2.6 Bảng chân lý của `control` cho 28 lệnh. 13
Bảng 2.7 Hai cách giải mã cho `alu_ctl`. 14

Bảng A.1 Phân công nhiệm vụ và đóng góp 22

LỜI GIỚI THIỆU

TODO: Long viết (1 trang). Nêu: bài toán (đồng hồ chuẩn ms), đề tài, phạm vi **chỉ mô phỏng** (không board, không Fmax), lời cảm ơn giảng viên. Không hứa điều báo cáo không chứng minh.

Hà Nội, ngày … tháng 12 năm 2026

Nhóm …

CHƯƠNG 1. GIỚI THIỆU

1.1 Bài toán

Đề tài thiết kế **WatchChip**: con chip của một chiếc đồng hồ điện tử kiểu Casio F-91W, viết bằng Verilog. Chip có CPU MIPS32 tự viết, bộ đếm thời gian, hiển thị LED 7 đoạn và VGA, còi, UART và ngắt. CPU có hai bản: **G1** chạy đơn chu kỳ, **G2** chạy đường ống 5 tầng. Cả hai chạy cùng một tập 28 lệnh và cùng một firmware.

Đồng hồ có năm chế độ: xem giờ, chỉnh giờ, bấm giờ, hẹn giờ (đếm xuống 0 rồi đếm tiếp sang số âm) và báo thức. Giờ có thể đồng bộ với máy tính qua UART.

Câu hỏi xuyên suốt của đồ án là: *muốn đồng hồ chuẩn tới 1 ms thì phần nào phải là phần cứng?* Phép tính mở đầu rất ngắn. Để sai không quá 1 ms sau 1 giờ (3600 s), tần số phải sai không quá $1\,\text{ms}/3600\,\text{s} = 0{,}28$ ppm. Một thạch anh thông thường sai cỡ ± 50 ppm (giả định, chưa kiểm datasheet), tức lệch $50 \times 10^{-6} \times 3600 = 0{,}18$ s mỗi giờ. Vì vậy chip cần hiệu chỉnh tần số theo giờ chuẩn từ máy tính (thanh ghi extttCAL) và cần bộ đếm phần cứng. Câu trả lời bằng số đo nằm ở mục 3.5.

Phạm vi. Đề tài chỉ chạy trong mô phỏng, không nạp lên board. Để mô phỏng nhanh, 1 ms của chip được rút thành $G_N = 50$ nhịp xung hệ thống thay vì 50 000. Mục 2.1 nói rõ hệ quả của hai điều này.

1.2 Cơ sở lý thuyết: kiến trúc tập lệnh MIPS

1.2.1 Tổng quan MIPS ISA

Kiến trúc tập lệnh (ISA) là hợp đồng giữa phần mềm và phần cứng: lệnh nào có, mã hóa ra sao, mỗi lệnh làm gì. MIPS là ISA kiểu RISC: lệnh dài cố định 32 bit, chỉ `lw` và `sw` mới đụng bộ nhớ, mọi phép tính khác lấy toán hạng từ thanh ghi.

WatchChip dùng một tập con gồm 28 lệnh của MIPS32, cộng một lệnh riêng `bcd` (mục 1.2.3). Ba điểm khác với MIPS đầy đủ cần nhớ:

- không có *delay slot*: lệnh ngay sau nhánh hoặc nhảy chỉ chạy khi nhánh không nhảy;
- không có ngoại lệ tràn số, lệnh lạ hay địa chỉ sai; nguồn duy nhất vào vector `0x180` là ngắt;
- bộ nhớ kiểu Harvard: bộ nhớ lệnh (IMEM) và bộ nhớ dữ liệu (DMEM) tách riêng.

1.2.2 Tập thanh ghi

Có 32 thanh ghi 32 bit `$0`–`$31`; `$0` luôn đọc ra 0 và ghi vào bị bỏ. `jal` ghi địa chỉ quay về vào `$31` (`$ra`). Hai thanh ghi `HI` và `LO` giữ kết quả của `div`. Ba thanh ghi của bộ đồng xử lý CP0 dùng cho ngắt: `$12` Status, `$13` Cause, `$14` EPC (mục 1.3.4). Bộ đếm chương trình PC dài 32 bit, địa chỉ tính theo byte, lệnh luôn căn 4.

1.2.3 Định dạng lệnh và 28 lệnh được hỗ trợ

Mọi lệnh dài 32 bit và thuộc một trong ba khuôn chính R, I, J. Hai nhóm `SPECIAL2` và `COP0` dùng lại khuôn R với vài trường được gán nghĩa riêng.

Khuôn	[31:26]	[25:21]	[20:16]	[15:11]	[10:6]	[5:0]
R	`op` = 0	`rs`	`rt`	`rd`	`shamt`	`funct`
I	`op`	`rs`	`rt`	`imm` (16 bit)		
J	`op`	`target` (26 bit)				
SPECIAL2	`0x1C`	`rs`	`rt`	`rd`	0	`funct`
COP0	`0x10`	`sub`	`rt`	`rd`	0	0 hoặc `0x18`

Bảng 1.1: Các khuôn lệnh của WatchChip.

Ký hiệu: `R[x]` là thanh ghi, `M[a]` là một word của bộ nhớ, `s(x)` hiểu số có dấu, `u(x)` hiểu số không dấu. `SE(imm)` là mở rộng dấu 16 lên 32 bit, `ZE(imm)` là mở rộng bằng 0. Đích nhánh là $\texttt{PC} + 4 + (\texttt{SE(imm)} \ll 2)$; đích `j` và `jal` là 4 bit cao của $\texttt{PC} + 4$ ghép với `target` và hai bit 0. Phép cộng và trừ lấy 32 bit thấp, không bẫy tràn. Trong `sub` của COP0, `0x00` là `mfc0`, `0x04` là `mtc0`, `0x10` là `eret`.

Tập lệnh gồm 12 lệnh khuôn R, 9 lệnh khuôn I, 2 lệnh khuôn J, 2 lệnh `SPECIAL2` (`mul`, `bcd`) và 3 lệnh COP0, tổng 28. Bảng 1.2 liệt kê từng lệnh.

Bảng 1.2: 28 lệnh của WatchChip: mã hóa và ý nghĩa.

#	Lệnh	op / funct	Ý nghĩa
1	`addu rd,rs,rt`	00/0x21	`R[rd] = R[rs] + R[rt]`
2	`subu rd,rs,rt`	00/0x23	`R[rd] = R[rs] − R[rt]`
3	`and rd,rs,rt`	00/0x24	`R[rd] = R[rs] & R[rt]`
4	`or rd,rs,rt`	00/0x25	`R[rd] = R[rs] | R[rt]`
5	`xor rd,rs,rt`	00/0x26	`R[rd] = R[rs] ^ R[rt]`
6	`slt rd,rs,rt`	00/0x2A	`R[rd] = s(R[rs]) < s(R[rt]) ? 1 : 0`
7	`sll rd,rt,sa`	00/0x00	`R[rd] = R[rt] << sa` (`sll $0,$0,0` = `nop` = 0x00000000)

#	Lệnh	op / funct	Ý nghĩa
8	`sra rd,rt,sa`	00/0x03	`R[rd] = s(R[rt]) >> sa` (dịch số học, chép bit dấu)
9	`jr rs`	00/0x08	`PC = R[rs]` (2 bit thấp bị bỏ)
10	`div rs,rt`	00/0x1A	Chia có dấu: `LO` = thương làm tròn về 0, `HI` = dư (cùng dấu với số bị chia). Chạy 33 chu kỳ CPU (một chu kỳ start, 32 bước chia).
11	`mfhi rd`	00/0x10	`R[rd] = HI`
12	`mflo rd`	00/0x12	`R[rd] = LO`
13	`addiu rt,rs,imm`	0x09	`R[rt] = R[rs] + SE(imm)`
14	`slti rt,rs,imm`	0x0A	`R[rt] = s(R[rs]) < s(SE(imm)) ? 1 : 0`
15	`andi rt,rs,imm`	0x0C	`R[rt] = R[rs] & ZE(imm)`
16	`ori rt,rs,imm`	0x0D	`R[rt] = R[rs] | ZE(imm)`
17	`lui rt,imm`	0x0F	`R[rt] = imm << 16`
18	`lw rt,imm(rs)`	0x23	`R[rt] = M[(R[rs] + SE(imm)) & ~3]`
19	`sw rt,imm(rs)`	0x2B	`M[(R[rs] + SE(imm)) & ~3] = R[rt]`
20	`beq rs,rt,L`	0x04	Nếu `R[rs] == R[rt]` thì `PC = PC + 4 + (SE(imm) << 2)`.
21	`bne rs,rt,L`	0x05	Nếu `R[rs] != R[rt]` thì `PC = PC + 4 + (SE(imm) << 2)`.
22	`j L`	0x02	Nhảy tới địa chỉ ghép từ `target` (xem trên).
23	`jal L`	0x03	`R[31] = PC + 4`, rồi nhảy như `j`.
24	`mul rd,rs,rt`	0x1C/0x02	`R[rd]` = 32 bit thấp của tích có dấu `s(R[rs]) × s(R[rt])`. `HI`, `LO` không đổi.
25	`bcd rd,rs`	0x1C/0x10	Nếu `u(R[rs])` $\leq$ 99 999 999 thì `R[rd]` gồm 8 chữ số thập phân, mỗi chữ số 4 bit, hàng đơn vị ở bit [3:0]; ngược lại `R[rd] = 0xFFFFFFFF`.
26	`mfc0 rt,rd`	0x10/rs = 0x00	`R[rt] = CP0[rd]`
27	`mtc0 rt,rd`	0x10/rs = 0x04	`CP0[rd] = R[rt]` (chỉ ghi các bit ghi được).
28	`eret`	0x42000018	`PC = EPC`, `Status.EXL = 0`

1.3 Cơ sở thiết kế

1.3.1 CPU đơn chu kỳ

CPU đơn chu kỳ làm xong một lệnh trong một chu kỳ: lấy lệnh, giải mã, đọc thanh ghi, tính, truy cập bộ nhớ, ghi kết quả. CPI (số chu kỳ trên một lệnh) bằng 1. Đổi lại, chu kỳ xung phải dài bằng đường đi chậm nhất, thường là của `lw`, nên mọi lệnh đều phải chờ theo nó. Thiết kế này dễ kiểm nhất và là cơ sở để so sánh. WatchChip hiện thực nó thành G1, với một sửa đổi cho bộ nhớ trong FPGA (mục 2.3).

1.3.2 Kỹ thuật đường ống (pipelining)

Đường ống chia lệnh thành các tầng và cho nhiều lệnh chạy chồng nhau, mỗi lệnh ở một tầng khác nhau. Năm tầng cổ điển là IF (lấy lệnh), ID (giải mã, đọc thanh ghi), EX (tính), MEM (truy cập bộ nhớ) và WB (ghi thanh ghi). Giữa hai tầng có một thanh ghi tầng giữ kết quả dở dang. Khi đầy ống, mỗi chu kỳ xong một lệnh, và chu kỳ xung chỉ cần dài bằng tầng chậm nhất, ngắn hơn nhiều so với đơn chu kỳ. WatchChip hiện thực nó thành G2.

1.3.3 Các loại xung đột và kỹ thuật xử lý

Xung đột (hazard) là tình huống lệnh kế tiếp không thể chạy ngay ở chu kỳ kế. Có ba loại:

- **Xung đột cấu trúc**: hai lệnh cần cùng một tài nguyên phần cứng trong cùng chu kỳ, ví dụ cùng đọc một bộ nhớ để lấy lệnh và để lấy dữ liệu. Tách IMEM và DMEM (kiểu Harvard) loại bỏ trường hợp này.
- **Xung đột dữ liệu**: lệnh sau cần kết quả lệnh trước chưa kịp ghi (RAW, đọc sau ghi). Kỹ thuật xử lý là *forwarding*: đưa kết quả thẳng từ đầu ra một tầng về đầu vào ALU, không đợi ghi vào thanh ghi. Riêng `lw` có dữ liệu muộn hơn: lệnh dùng ngay kết quả của nó phải *đứng* (stall) một chu kỳ. Đó là xung đột load-use.
- **Xung đột điều khiển**: chưa biết nhánh có nhảy hay không thì chưa biết lệnh kế là lệnh nào. Cách đơn giản nhất là dự đoán không nhảy; nếu đoán sai thì hủy (flush) các lệnh đã lấy nhầm.

Chương 2 nói từng loại được xử lý thế nào trong G2.

1.3.4 Ngắt và CP0

Ngắt cho ngoại vi (ở đây là nút bấm, UART) chen vào chương trình đang chạy mà không cần CPU hỏi liên tục. Khi nhận ngắt, CPU lưu địa chỉ lệnh cần quay về vào `EPC`, bật cờ `EXL` để chặn

ngắt lồng, rồi nhảy tới địa chỉ cố định `0x180`. Trình xử lý kết thúc bằng `eret`: PC lấy lại từ `EPC` và `EXL` xóa về 0. Các thanh ghi `Status`, `Cause`, `EPC` nằm trong CP0 và đọc, ghi bằng `mfc0`, `mtc0`. Ngắt *chính xác* nghĩa là lệnh nào đã qua một điểm nhất định thì chạy trọn, lệnh chưa qua thì bị hủy và chạy lại sau `eret`; điều này quan trọng với đường ống, nơi nhiều lệnh dở dang cùng lúc.

TODO: Long. Chèn tối đa 3 hình từ sách Patterson & Hennessy cho mục 1.3 (đường dữ liệu đơn chu kỳ, đường ống 5 tầng, forwarding), mỗi hình ghi "nguồn: P&H Fig x.y". Báo cáo chỉ được dùng hình sách ở chương 1.

CHƯƠNG 2. PHƯƠNG PHÁP LUẬN VÀ THIẾT KẾ

2.1 Công cụ và cấu trúc mã nguồn

2.1.1 Phạm vi của đề tài

Bốn điều sau đây áp dụng cho mọi số liệu trong báo cáo.

- **Chỉ mô phỏng.** Không có board FPGA, không có chân vật lý. Mọi kết quả đến từ mô phỏng bằng Icarus Verilog, và ModelSim cho G2.
- **Thời gian rút gọn.** Tham số `G_N` là số xung hệ thống trong 1 ms. Thật là 50 000 (xung 50 MHz); trong mô phỏng đặt $G_N = 50$, nên 1 ms của chip chỉ tốn 50 xung, ít hơn thật đúng 1000 lần. Số đo thời gian trong báo cáo vì thế tính theo chu kỳ hoặc theo ms *của chip rút gọn*, không phải giây đồng hồ treo tường.
- **Không có Fmax.** Nhóm không có Vivado hay Quartus, nên không có bước đặt và nối dây (place and route) và không có tần số tối đa. Số ô logic (LUT, FF), nếu được nêu, lấy từ Yosys và chỉ là ước lượng cấu trúc, không thay cho kết quả của công cụ hãng.
- **G2 chỉ chạy trong hồi quy.** Bản demo chạy lõi G1. Lõi G2 chạy trong co-sim với ISS, trong ModelSim và trong các chương trình kiểm hazard, không chạy trong bản demo.

Phần lớn mã nguồn do công cụ AI viết theo yêu cầu của nhóm. Mỗi thành viên đã đọc phần mã của mình, chạy lại các testbench liên quan và phải giải thích được nó.

2.1.2 Công cụ

Bảng 2.1 liệt kê công cụ, phiên bản (ghi trong `evidence/20261001_wp0_long.log`) và việc nó làm trong đề tài.

TODO: TV2. Điền phiên bản ModelSim và Python (hai ô còn trống trong Bảng 2.1); chạy `vsim -version` và `python --version` trên máy đã chạy hồi quy.

2.1.3 Cây thư mục

Từ thư mục gốc của project:

- `src/`: 21 file Verilog của chip (CPU, ALU, bộ đếm, hiển thị, bus, UART);
- `tb/`: testbench, kèm `tb/vectors/` là các file vector kiểm;

Công cụ	Phiên bản	Dùng để
Icarus Verilog	14.0	Biên dịch và chạy testbench (`iverilog`, `vvp`) cho mọi khối và cho G1.
ModelSim	*chưa ghi*	Chạy lại G2 (`tools/run_modelsim_g2.py`) và so trace với Icarus.
Yosys	0.69+173	Tổng hợp để đếm LUT và FF (ước lượng, không phải Vivado).
MARS	4.5	Bộ mô phỏng MIPS độc lập, làm chuẩn so trace để không cùng sai với ISS tự viết.
Python	*chưa ghi*	`asm.py` dịch hợp ngữ, `iss.py` mô phỏng mức lệnh (ISS), `cmp_trace.py` so trace, `cosim_iss.py` và `cosim_mars.py` chạy co-sim; `pytest` cho các script.

Bảng 2.1: Công cụ dùng trong đề tài.

- `fw/`: firmware hợp ngữ MIPS (`main.s`) và các chương trình kiểm trong `fw/tests/`;
- `tools/`: assembler, ISS, co-sim, script chạy hồi quy;
- `docs/`: tài liệu hợp đồng (tập lệnh, bảng điều khiển, luật pipeline, bản đồ MMIO);
- `evidence/`: log chạy thật, là nguồn của mọi con số trong báo cáo;
- `work/`: đầu ra tạm của mô phỏng;
- `report/`: báo cáo này.

2.1.4 Cách chạy testbench

Cần Git Bash hoặc bash. Từ thư mục gốc:

```
bash tools/setup_env.sh        # cài công cụ, một lần
source tools/env.sh            # mỗi lần mở terminal mới
bash tools/run_unit.sh alu     # chạy tb/alu_tb.v
bash tools/run_unit.sh         # chạy cả hồi quy
```

Mỗi testbench in dòng cuối `PASS <tên> <số lần kiểm>` hoặc `FAIL`. Ví dụ, `alu_tb` cho `PASS alu_tb 4200`. Hồi quy đầy đủ gần nhất (`evidence/20261027_regress_rbuf.log`) có 41 dòng PASS và kết thúc bằng `EXIT 0`.

Hai testbench `cpu_g1_tb` và `cpu_pipe_tb` cần có chương trình nạp vào IMEM (đối số `+imem`), nên chạy riêng bằng `run_unit.sh` sẽ báo FAIL. Chúng chạy qua co-sim:

```
python tools/cosim_iss.py          # G1
python tools/cosim_iss.py --g2     # G2
```

2.2 Thông số và cổng của CPU

G1 (`src/cpu_g1.v`, dòng 22–43) và G2 (`src/cpu_pipe.v`, dòng 26–46) có *cùng một bộ cổng*, nên SoC thay lõi này bằng lõi kia mà không đổi dây. Bảng 2.2 liệt kê tham số, cổng và bộ nhớ.

Tên	Hướng	Rộng	Ý nghĩa
Tham số			
`IMEM_FILE`, `DMEM_FILE`	—	chuỗi	File hex nạp vào IMEM, DMEM; rỗng thì không nạp.
`G_IMEM_AW`	—	số	IMEM có $2^{\texttt{G_IMEM_AW}}$ word; mặc định 10 (1024 word).
`G_BCD_COMB`	—	0/1	0: `bcd` chạy tuần tự nhiều chu kỳ; 1: tổ hợp.
`TRACE_FILE`	—	chuỗi	File ghi trace từng lệnh (chỉ mô phỏng).
Cổng			
`clk`	vào	1	Xung hệ thống.
`rst`	vào	1	Reset đồng bộ.
`cpu_ce`	vào	1	Cho phép CPU bước một nhịp (mục 2.3).
`bus_addr`	ra	8	Địa chỉ thanh ghi ngoại vi, căn 4.
`bus_wdata`	ra	32	Dữ liệu `sw` gửi ngoại vi.
`bus_we`, `bus_re`	ra	1	Ghi, đọc ngoại vi.
`bus_rdata`	vào	32	Dữ liệu ngoại vi trả cho `lw`.
`irq`	vào	1	Yêu cầu ngắt, tính theo mức.
`irq_ip`	vào	8	Cờ ngắt đang chờ, đọc được bằng `mfc0 $13`.
`div_dz`	ra	1	Cờ chia cho 0.
`pc_out`	ra	32	PC hiện tại, để testbench quan sát.
`retire`	ra	1	Xung báo một lệnh vừa xong.
Bộ nhớ trong lõi			
`imem`	—	$2^{10} \times 32$	Bộ nhớ lệnh, kiểu Harvard.
`dmem`	—	1024×32	Bộ nhớ dữ liệu 4 KB, địa chỉ `0x0000_0000`–`0x0000_0FFF`.

Bảng 2.2: Tham số, cổng và bộ nhớ của CPU (giống nhau ở G1 và G2).

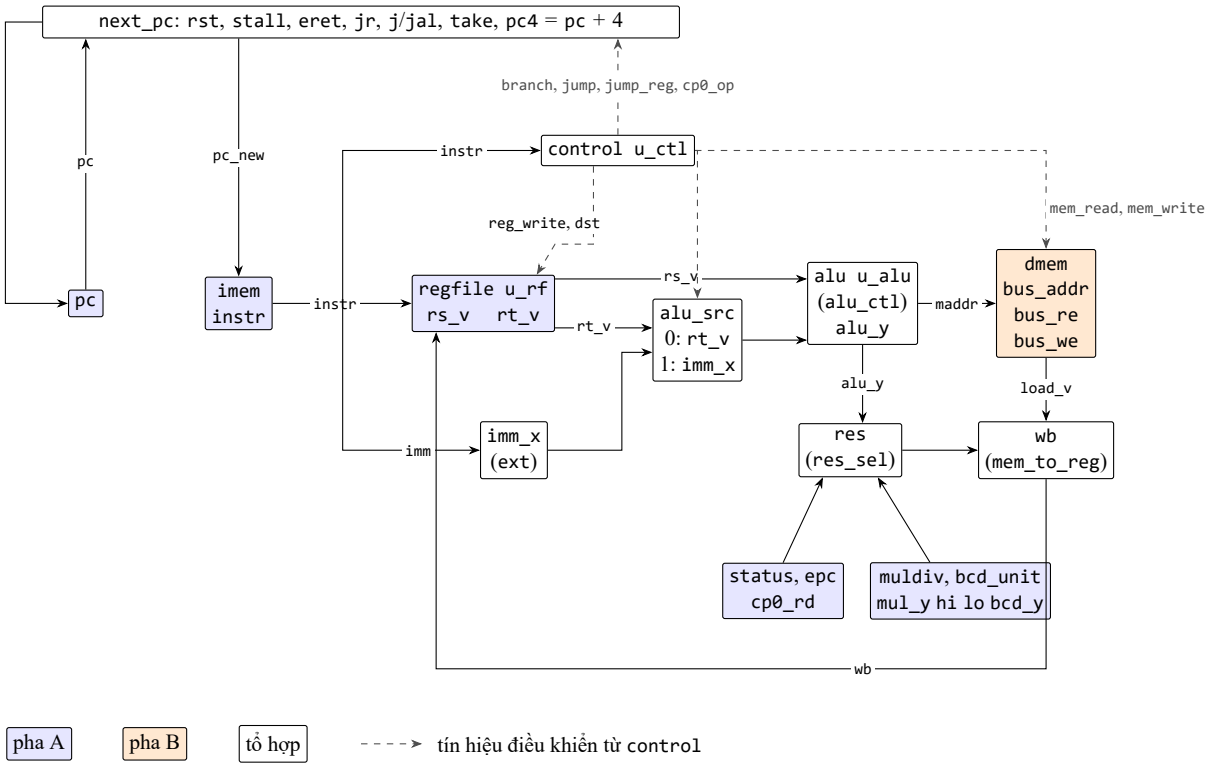
Địa chỉ `lw`/`sw` nằm trong vùng `0xFFFF_FF00`–`0xFFFF_FFFF` thì đi ra bus tới ngoại vi (MMIO). Chương trình truy cập vùng này bằng offset âm so với `$zero`, ví dụ `lw $t0, -252($zero)`. Địa chỉ ở nơi khác thì đọc ra 0 và ghi bị bỏ. `lw`/`sw` lệch 4 byte thì bỏ hai bit thấp, không có ngoại lệ căn lề.

2.3 CPU G1: đơn chu kỳ

2.3.1 Datapath

Hình 2.1 là đường dữ liệu của G1. Dữ liệu đi từ trái sang phải theo bảy bước.

1. `pc` giữ địa chỉ lệnh; IMEM (`imem`) trả lệnh `instr`.
2. Khối `control u_ctl` giải mã `instr` thành các tín hiệu điều khiển (mục 2.4); đường nét đứt trên hình.
3. Thanh ghi (`regfile u_rf`) đọc hai giá trị `rs_v`, `rt_v`. Khối `imm_x` mở rộng hằng 16 bit của lệnh.
4. ALU (`alu u_alu`) nhận `rs_v` và một trong hai nguồn: `rt_v` hoặc `imm_x` (chọn bằng `alu_src`), rồi ra `alu_y`.
5. Với `lw` và `sw`, `alu_y` là địa chỉ (`maddr`). Bộ nhớ dữ liệu hoặc ngoại vi trả `load_v`.
6. Khối `res` chọn kết quả theo `res_sel`: ALU, `mul_y`, `hi`, `lo`, `bcd_y`, `cp0_rd` hoặc `pc + 4`. Khối `wb` chọn giữa `load_v` và `res` (`mem_to_reg`) rồi ghi về `regfile`.
7. Khối `next_pc` tính địa chỉ lệnh kế theo thứ tự ưu tiên: reset, đứng chờ (`stall`), `eret`, `jr`, `j`/`jal`, rẽ nhánh (`take`), cuối cùng là `pc4`. Vào ngắt thì PC bị thay bằng `0x180`: trong code, `pc_new` bằng `0x180` khi có ngắt và bằng `next_pc` khi không (`src/cpu_g1.v`, dòng 167).



Hình 2.1: Datapath CPU G1. Nền xanh: khối cập nhật ở pha A; nền cam: khối làm việc ở pha B; trắng: mạch tổ hợp; nét đứt: tín hiệu điều khiển từ `control`. Mọi nhãn là tên tín hiệu hoặc module trong `src/cpu_g1.v`.

Hai đoạn code dưới đây là phần của hình mà người đọc hay hỏi nhất. Đoạn đầu tạo `bus_re`, `bus_we` và `load_v`; đoạn sau là thứ tự ưu tiên của `next_pc`.

Mã 2.1: Địa chỉ bộ nhớ, tín hiệu bus và dữ liệu `lw` (`src/cpu_g1.v`).

```
122     wire [31:0] maddr   = alu_y;
123     wire        in_dmem = (maddr[31:12] == 20'd0);
124     wire        in_mmio = (maddr[31:8] == 24'hFFFFFF);
125     assign bus_addr  = {maddr[7:2], 2'b00};
126     assign bus_wdata = rt_v;
127     assign bus_we    = ph_b && mem_write && in_mmio;
128     assign bus_re    = ph_b && mem_read  && in_mmio;
129
130     wire [31:0] load_v = (lsel == 2'b01) ? dmem_q : (lsel == 2'b10) ? bus_rdata : 32'd0;
        ↪  // dữ liệu lw đọc được
```

Mã 2.2: Thứ tự ưu tiên của `next_pc` (`src/cpu_g1.v`).

```
152     reg  [31:0] next_pc;
153     // Thứ tự ưu tiên (từ trên xuống): reset → đứng chờ → eret → jr → j/jal → rẽ nhánh →
        ↪  đi tiếp +4
154     always @* begin
155         if (rst)                  next_pc = 32'd0;
156         else if (stall)           next_pc = pc;
157         else if (cp0_op == 2'b10) next_pc = epc;
158         else if (jump_reg)        next_pc = {rs_v[31:2], 2'b00};
159         else if (jump)            next_pc = {pc4[31:28], instr[25:0], 2'b00};
160         else if (take)            next_pc = pc4 + {imm_x[29:0], 2'b00};
161         else                      next_pc = pc4;
162     end
```

2.3.2 Một chu kỳ CPU gồm hai pha

Mỗi chu kỳ CPU của G1 gồm hai cạnh `cpu_ce` liên tiếp, gọi là **pha A** (đầu chu kỳ) và **pha B** (giữa chu kỳ). Hai pha dùng cùng một xung `clk`; không có xung thứ hai.

Lúc	Việc xảy ra
Pha A	`pc`, `regfile`, `HI`/`LO`, CP0 cập nhật. IMEM chốt `imem[pc_new]`; thanh ghi đầu ra của IMEM từ đây là lệnh của chu kỳ mới.
Giữa A và B	Giải mã, đọc `regfile`, ALU tính địa chỉ hoặc kết quả.
Pha B	DMEM và MMIO chốt địa chỉ: `sw` ghi, `lw` đọc vào thanh ghi đầu ra. Tác dụng phụ của ngoại vi xảy ra ở đây (`bus_re`, `bus_we`).
Giữa B và A	Dữ liệu `lw` đi qua `wb` về cổng ghi của `regfile`; `next_pc` được tính.

Bảng 2.3: Việc của từng pha trong một chu kỳ CPU của G1.

Để nhớ: *B (bộ nhớ) trước, A (cập nhật) sau*. Sau reset, `phb = 1` nên cạnh `cpu_ce` đầu tiên là pha B, và IMEM đã giữ sẵn lệnh ở địa chỉ 0 (`imem[0]`). Khi lệnh nhiều chu kỳ (`div`, `bcd`) đang chạy, `next_pc = pc` và IMEM chốt lại đúng lệnh cũ.

2.3.3 Đường đi của `lw $t0, -252($zero)`

Offset -252 mở rộng dấu thành `0xFFFFFF04`, nằm trong vùng ngoại vi, tại thanh ghi `MS_OF_DAY` (số ms trong ngày). Đọc thanh ghi này đồng thời chụp lại ngày vào `DAY_SHADOW`; firmware đọc `MS_OF_DAY` trước rồi mới đọc `DAY_SHADOW`, để cặp (ms, ngày) khớp nhau kể cả khi nửa đêm qua giữa hai lần đọc.

Bước	Lúc	Việc
1	Pha A	Thanh ghi đầu ra IMEM đã chứa `lw`, vì IMEM chốt `imem[pc_new]`.
2	Giữa A và B	`control` bật `mem_read`, `mem_to_reg`, `reg_write`; `regfile` đọc `$zero` = 0; ALU cộng $0 + $ `0xFFFFFF04` ra `maddr`.
3	Pha B	`bus_re = ph_b && mem_read && in_mmio` (dòng 128) bằng 1; ngoại vi chốt `bus_addr` = `0x04`, trả `MS_OF_DAY` và chụp ngày.
4	Giữa B và A	`load_v` lấy `bus_rdata`; `wb` chọn `load_v` (dòng 130, `mem_to_reg` = 1).
5	Pha A kế tiếp	`regfile` ghi `wb` vào `$t0`.

Bảng 2.4: Đường đi của một lệnh `lw` đọc ngoại vi trong G1.

Ở G2 (`src/cpu_pipe.v`), cùng lệnh đó đi qua năm tầng: IF lấy lệnh, ID giải mã và đọc `$zero`, EX tính địa chỉ, MEM đọc bus đúng một bước, WB ghi `$t0`. Lệnh ngay sau mà dùng `$t0` phải đứng một chu kỳ (load-use).

2.3.4 Vì sao chọn: G1 hai pha

Vấn đề. Bộ nhớ trong FPGA (BRAM) đọc *đồng bộ*: địa chỉ phải được chốt ở một cạnh xung, dữ liệu mới có ở cạnh sau. Một lệnh `lw` cần hai lần truy cập nối tiếp: lấy lệnh từ IMEM, rồi lấy dữ liệu từ DMEM theo địa chỉ ALU vừa tính.

Chọn (c) vì nó giữ đúng ý đồ "BRAM ở sườn xuống" của thiết kế ngày 24/09 nhưng dùng hai cạnh lên, nên không bị ràng buộc nửa chu kỳ và không thêm xung. Điểm sửa so với bản 24/09: IMEM phải được địa chỉ bằng `next_pc` ở pha A, để thanh ghi đầu ra của nó là lệnh của chu kỳ sau. Phần "IMEM suy ra BRAM" chưa kiểm vì không có công cụ hãng.

Cách kiểm. G1 chạy chương trình MIPS rồi so từng lệnh với ISS và với MARS. Hồi quy gần nhất có `PASS cosim_mars 19` và `PASS cosim_iss 25`, trong đó có chương trình `lw` đọc MMIO (`evidence/20261027_regress_rbuf.log`).

Phương án	Ưu	Nhược
(a) BRAM chốt ở cạnh xuống, một pha (thiết kế ngày 24/09)	Chỉ cần một chu kỳ.	Nếu IMEM chốt `pc` ở giữa chu kỳ thì không còn chỗ cho DMEM: địa chỉ của `lw` chưa tính xong. Mỗi nửa chu kỳ chỉ còn 10 ns.
(b) Bộ nhớ đọc tổ hợp (LUT-RAM), một pha	Không cần pha.	Suy luận (chưa kiểm vì không có Vivado): chỉ hợp với bộ nhớ nhỏ, vì bộ nhớ lớn trên FPGA Xilinx là BRAM đọc đồng bộ.
(c) Hai cạnh `cpu_ce` cùng một `clk` (chọn)	Mỗi nửa có trọn 20 ns; không có xung thứ hai; IMEM đọc đồng bộ nên viết được dưới dạng BRAM.	Mỗi chu kỳ CPU tốn hai xung hệ thống, nên G1 chậm hơn đơn chu kỳ lý tưởng.

Bảng 2.5: Ba cách tổ chức bộ nhớ cho G1.

2.4 Khối điều khiển và bảng chân lý

Khối `control` (`src/control.v`) nhận một lệnh 32 bit và trả ra 16 tín hiệu điều khiển, hoàn toàn bằng mạch tổ hợp. G1 và G2 dùng chung khối này. Ý nghĩa các tín hiệu:

- `reg_write`: lệnh có ghi thanh ghi đích; `dst`: đích là `rd`, `rt` hay `$31`;
- `alu_src`: toán hạng B của ALU là `rt_v` (0) hay `imm_x` (1); `ext`: hằng 16 bit mở rộng dấu (SE), mở rộng 0 (ZE) hay đặt lên 16 bit cao (LUI);
- `alu_ctl`: phép của ALU (ADD, SUB, AND, OR, XOR, SLT, SLL, SRA); `res_sel`: kết quả đến từ ALU, MUL, HI, LO, BCD, CP0 hay PC4;
- `mem_read`, `mem_write`, `mem_to_reg`: đọc, ghi bộ nhớ dữ liệu, và kết quả ghi về thanh ghi lấy từ bộ nhớ;
- `branch`: không, `beq` hoặc `bne`; `jump`: `j`/`jal`; `jump_reg`: `jr`;
- `multi`: lệnh nhiều chu kỳ, không, `div` hoặc `bcd` (bản tuần tự); `cp0_op`: không, `mtc0` hoặc `eret`;
- `uses_rs`, `uses_rt`: lệnh có *thật sự* đọc `rs`, `rt`. Bộ phát hiện load-use của G2 dùng hai tín hiệu này để khỏi đứng oan.

Bảng 2.6 cho giá trị của 16 tín hiệu ở cả 28 lệnh. Ô trống bằng 0; `--` là không dùng (khi đó `dst` bị ép về 0). Opcode hoặc funct không có trong bảng cho mọi tín hiệu bằng 0, tức lệnh lạ chạy như `nop`; cổng phụ `valid` chỉ để mô phỏng báo "lệnh lạ" và không thuộc 16 tín hiệu.

Bảng 2.6: Bảng chân lý của `control` cho 28 lệnh.

#	Lệnh	reg_write	dst	alu_src	ext	alu_ctl	res_sel	mem_read	mem_write	mem_to_reg	branch	jump	jump_reg	multi	cp0_op	uses_rs	uses_rt
1	addu	1	rd	0	–	ADD	ALU									1	1
2	subu	1	rd	0	–	SUB	ALU									1	1
3	and	1	rd	0	–	AND	ALU									1	1
4	or	1	rd	0	–	OR	ALU									1	1
5	xor	1	rd	0	–	XOR	ALU									1	1
6	slt	1	rd	0	–	SLT	ALU									1	1
7	sll	1	rd	0	–	SLL	ALU									0	1
8	sra	1	rd	0	–	SRA	ALU									0	1
9	jr	0	–	–	–	–	–						1			1	0
10	div	0	–	–	–	–	–							div		1	1
11	mfhi	1	rd	–	–	–	HI									0	0
12	mflo	1	rd	–	–	–	LO									0	0
13	addiu	1	rt	1	SE	ADD	ALU									1	0
14	slti	1	rt	1	SE	SLT	ALU									1	0
15	andi	1	rt	1	ZE	AND	ALU									1	0
16	ori	1	rt	1	ZE	OR	ALU									1	0
17	lui	1	rt	1	LUI	OR	ALU									0	0
18	lw	1	rt	1	SE	ADD	ALU	1		1						1	0
19	sw	0	–	1	SE	ADD	–		1							1	1
20	beq	0	–	0	SE		–				beq					1	1
21	bne	0	–	0	SE		–				bne					1	1
22	j	0	–	–	–	–	–					1				0	0
23	jal	1	31	–	–	–	PC4					1				0	0
24	mul	1	rd	0	–	–	MUL									1	1
25	bcd	1	rd	–	–	–	BCD							bcd		1	0
26	mfc0	1	rt	–	–	–	CP0									0	0
27	mtc0	0	–	–	–	–	–								mtc0	0	1
28	eret	0	–	–	–	–	–								eret	0	0

Cách đọc một dòng. Lấy `lw` (dòng 18): `reg_write` = 1 và `dst` = `rt` (ghi vào `rt`); `alu_src` = 1, `ext` = SE, `alu_ctl` = ADD (cộng `rs` với hằng đã mở rộng dấu ra địa chỉ); `mem_read` = 1 và `mem_to_reg` = 1 (đọc bộ nhớ và ghi dữ liệu đó về `rt`); `uses_rs` = 1, `uses_rt` = 0 (`rt` là nơi ghi, không phải nơi đọc). Lệnh `sw` (dòng 19) khác ở chỗ `reg_write` = 0, `mem_write` = 1 và `uses_rt` = 1, vì `rt` là giá trị cần ghi ra.

Trong code, `lw` và `sw` là hai nhánh của một lệnh `case`:

Mã 2.3: Giải mã `lw` và `sw` (`src/control.v`).

```
87              6'h23: begin i_alu(A_ADD, E_SE, 1); mem_read = 1; mem_to_reg = 1; end   //
                ↪  lw: ALU tính địa chỉ, rồi đọc bộ nhớ
88              6'h2B: begin                                                    // sw: ALU tính
                ↪  địa chỉ, rồi ghi bộ nhớ
89                  valid = 1; alu_src = 1; ext = E_SE; alu_ctl = A_ADD; mem_write = 1;
90                  uses_rs = 1; uses_rt = 1;
91              end
```

2.4.1 Vì sao chọn: giải mã một cấp

Vấn đề. Từ opcode và funct, `control` phải sinh ra phép cho ALU. Sách giáo khoa giải mã hai cấp; nhóm cân nhắc hai cách.

Phương án	Ưu	Nhược
(a) Hai cấp: bộ điều khiển chính sinh `ALUOp` 2 bit, khối ALU control sinh `alu_ctl` từ `ALUOp` và funct (như P&H)	Bảng chính nhỏ; đúng cách sách trình bày.	Hai khối, hai bảng phải khớp nhau. Nhóm có 4 nhóm opcode (R, I/J, SPECIAL2, COP0) nên khối thứ hai phải xử lý nhiều trường hợp.
(b) Một cấp: sinh thẳng `alu_ctl` từ opcode và funct (chọn)	Một bảng phẳng 28 dòng, kiểm một lần.	Khối `control` lớn hơn một chút.

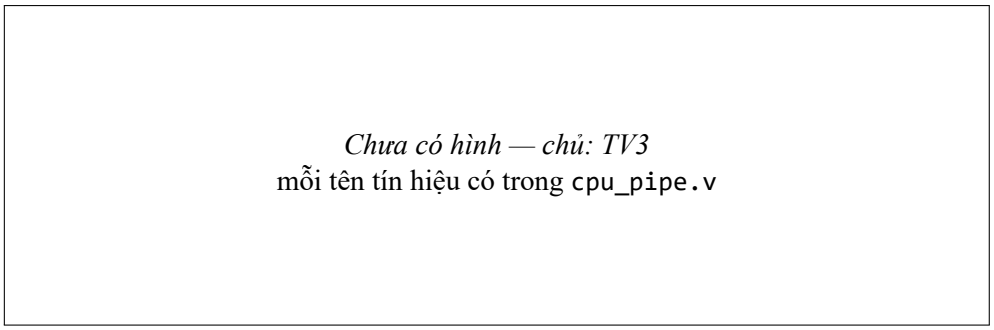
Bảng 2.7: Hai cách giải mã cho `alu_ctl`.

Chọn (b) vì với 28 lệnh, một bảng phẳng dễ đối chiếu với tài liệu hơn hai bảng nối tiếp. Cái giá là khối lớn hơn một chút; nhóm chưa đo mức chênh này vì không có công cụ tổng hợp của hãng.

Cách kiểm. Testbench `control_tb` so đầu ra với `tb/vectors/control.txt` cho cả 28 lệnh, rồi thử thêm lệnh ngẫu nhiên để kiểm lệnh lạ cho ra toàn 0; hồi quy ghi `PASS control_tb 82646`. Từng dòng của Bảng 2.6 cũng khớp với file vector đó, nên sửa `control.v` mà không sửa file vector thì `control_tb` báo FAIL.

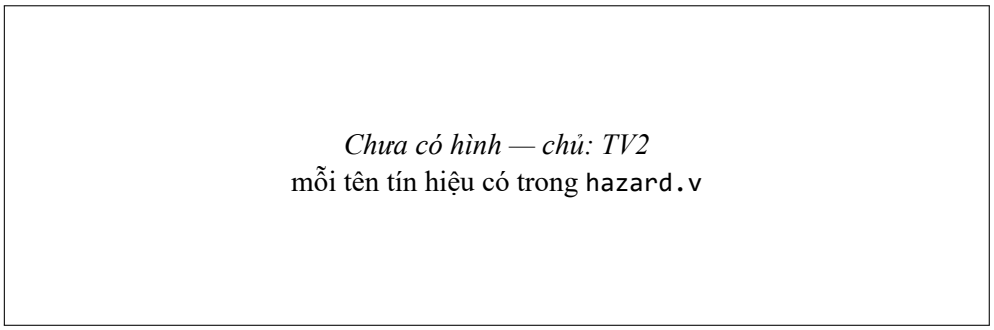
2.5 CPU G2: pipeline 5 tầng

TODO: TV3 + TV4. Từng tầng IF, ID, EX, MEM, WB và bốn thanh ghi tầng (trường, độ rộng), Bảng 2.4. Nguồn: `docs/pipeline-policy.md` mục 0–2, `src/cpu_pipe.v`.



Hình 2.2: Datapath CPU G2 đầy đủ (H3)

2.6 Xử lý xung đột



Hình 2.3: Khối forwarding và hazard unit (H4)

TODO: TV2 (TV1 phần `busy`, TV4 phần nhánh). Mỗi loại đi đủ 5 bước: định nghĩa, biểu thức điều kiện, khối phần cứng, đoạn code, bảng diễn biến theo chu kỳ. Gồm RAW ở EX/MEM, load-use, nhánh quyết định ở EX, `busy` khi `div`/`bcd`, `mtc0` sang `mfc0`, `eret`. Bảng chu kỳ khớp `docs/pipeline-policy.md` mục 7. Mục "vì sao chọn": nhánh ở EX hay ở ID, có ≥ 2 phương án.

2.7 ALU, nhân, chia và lệnh `bcd`

TODO: TV1 + TV4. ALU, `muldiv` (chia khôi phục 33 chu kỳ: một chu kỳ start và 32 bước), `bcd_unit` hai bản, Bảng 2.5. Nguồn: `docs/isa-spec.md` mục 3–4, `src/alu.v`, `src/muldiv.v`, `src/bcd_unit.v`.

2.8 CP0 và ngắt

TODO: TV5. CP0, ngắt ở G1 và ngắt chính xác ở G2: lệnh ở ID và ID/EX bị hủy, EPC. Ghi rõ hạn chế: kiểm thử ngắt mới có mẫu nhỏ. Nguồn: `docs/isa-spec.md` mục 5, `docs/pipeline-policy.md` mục 10.

Chưa có hình — chủ: TV5
khớp `docs/pipeline-policy.md` mục 10

Hình 2.4: Sơ đồ vào và ra ngắt (H5)

2.9 Đoạn code chính

TODO: Mỗi thành viên trích một đoạn $\leq$ 20 dòng có chú thích của lát mình, đúng file hiện tại (`\maunguon` với dòng đầu và dòng cuối).

2.10 Tổng thể SoC

Chưa có hình — chủ: TV3
khối trên hình = module trong `soc_top.v`

Hình 2.5: Sơ đồ tổng thể SoC WatchChip (H1)

TODO: TV3. Thông số (IMEM/DMEM, xung nhịp mô phỏng), giao diện vào/ra, bản đồ MMIO khớp `docs/mmio-map.md`.

2.11 Bộ điều khiển hiển thị

TODO: TV4. Display RAM, ROM ký tự, quét 7 đoạn (khoảng tắt `G_BLANK`), VGA đuổi tia. Khớp `docs/display-spec.md`.

2.12 Timebase, hiệu chỉnh CAL và các bộ định thời

TODO: TV5. Timebase và `CAL`, RTC, bấm giờ có capture, hẹn giờ, báo thức, nút.

2.13 Firmware

TODO: TV1. Năm chế độ, vòng lặp chính, đồng bộ giờ qua UART và `CAL`.

CHƯƠNG 3. KIỂM THỬ VÀ KẾT QUẢ

3.1 Chiến lược kiểm thử

TODO: TV2/TV4. Unit → co-sim ISS → bộ test hazard → `system_tb` (G1 và G2, có/không ngắt) → màn hình ảo; kiểm thử đột biến. Số kiểm lấy từ hồi quy mới nhất trong `evidence/`. Nháp màn ảo: `report/3.1-man-ao.md`.

3.2 Trace và dạng sóng

Chưa có hình — chủ: TV3
4 ảnh, chú thích chu kỳ khớp `pipeline-policy.md` §7

Hình 3.1: Dạng sóng: load-use bubble, flush nhánh, `busy` freeze, vào/ra ngắt (H8)

3.3 Hiệu năng G1 và G2: CPI tách theo nguyên nhân

TODO: TV2/TV3. Số đếm bằng `tools/cpi.py`, không "≈". Nháp: `report/3.3-cpi.md`.

3.4 Khảo sát thiết kế (DSE) và tài nguyên

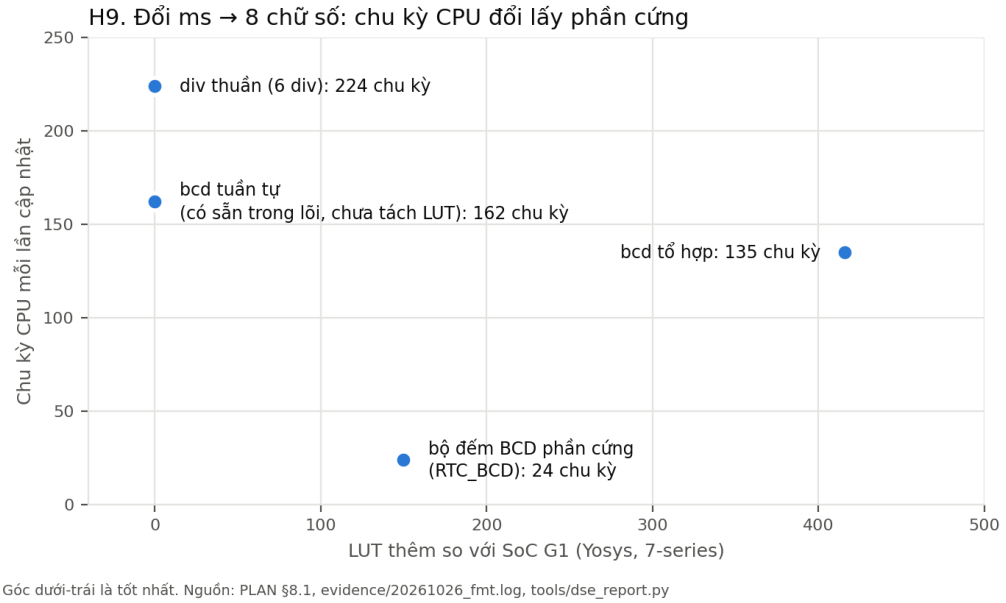
TODO: TV1. Nháp: `report/3.4-dse.md`; còn thiếu phần tài nguyên (plan §8.2).

3.5 Thí nghiệm E1′–E4′

TODO: Mỗi thí nghiệm: câu hỏi → cách đo → bảng/hình → kết luận + phạm vi đúng. Trả lời câu hỏi "chuẩn ms" của chương 1.

3.5.1 E1′, E2′: hiệu chỉnh CAL

TODO: TV5. Nháp: `report/2.4-cal-thi-nghiem.md`.



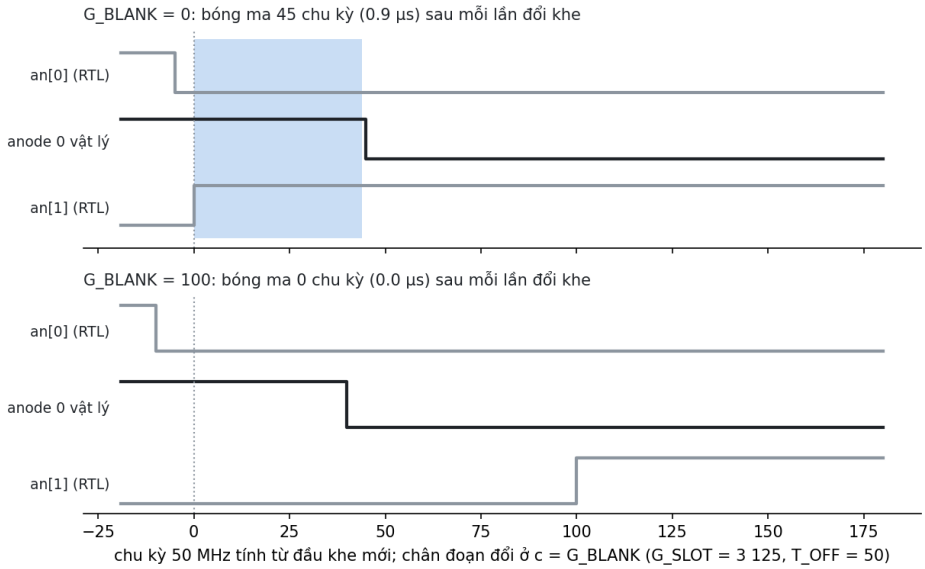
Hình 3.2: Chi phí đổi ms sang chữ số (H9)

3.5.2 E3: polling, ngắt và capture

TODO: TV3 + TV2. Nháp: `report/2.5-e3-thi-nghiem.md`.

3.5.3 E4′: bóng ma khi quét 7 đoạn

TODO: TV4. Nháp: `report/2.3-bongma-e4.md`. Kèm H10 (màn hình ảo).



Hình 3.3: Bóng ma theo thời gian tắt của chân đoạn, ở 3 giá trị `G_SLOT`

3.6 Tổng kết: đạt và chưa đạt

TODO: TV2. Nháp: `report/3.6-bang-dat.md`. Mỗi dòng trỏ về 1 file `evidence/`; người viết tự chạy lại rồi mới tick.

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết quả đạt được

TODO: Long. Tóm tắt có số, mỗi số trỏ về mục chương 3.

4.2 Hạn chế

TODO: Long. Lấy đúng cột CHƯA ĐẠT của bảng tổng kết, có số: chỉ mô phỏng, không Fmax, ≥ 3 câu UART liền nhau bị chụp giờ trễ, race PENDING chưa test tất định, …

4.3 Hướng phát triển

TODO: Long. Nạp board + đo thật; `sync_host.py` phía PC; hàng đợi vòng cho UART; Fmax khi có công cụ FPGA.

PHỤ LỤC A. PHÂN CÔNG CÔNG VIỆC

Bảng A.1: Phân công nhiệm vụ và đóng góp

STT	Họ và tên	MSSV	Gói phụ trách	%	Bằng chứng
1	Đoàn Danh Long		biên tập, …		
2	TV1		ALU & số học		
3	TV2		công cụ & hazard		
4	TV3		CPU & chip		
5	TV4		hiển thị & âm thanh		
6	TV5		định thời ms		

TODO: Long. Đóng góp ghi % (không ghi thang 1–5 như Nhóm 3), kèm bằng chứng (`hop-tuan.md`, kết quả cổng). Chốt cách khai phần AI viết (`KE-HOACH` §4b).

PHỤ LỤC B. KẾ HOẠCH VÀ TIẾN ĐỘ

TODO: Long. Bảng kế hoạch (`docs/goi-viec.md`) đặt cạnh thực tế (`docs/hop-tuan.md` + worklog), theo tuần.

PHỤ LỤC C. HƯỚNG DẪN CHẠY LẠI

TODO: TV2. Người khác chạy được trên máy họ. Tối thiểu:

```
source tools/env.sh
bash tools/run_unit.sh          # full regression
python tools/cosim_iss.py --g2 --rand 0
```

PHỤ LỤC D. TRÍCH TRACE VÀ LOG

TODO: TV2. Trích ngắn từ `evidence/`, ghi tên file gốc.

TÀI LIỆU THAM KHẢO

[1] D. A. Patterson, J. L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*, MIPS Edition. Morgan Kaufmann.

[2] MIPS Technologies. *MIPS32 Architecture for Programmers, Volume II: The MIPS32 Instruction Set*.

[3] MIPS Technologies. *MIPS32 Architecture for Programmers, Volume III: The MIPS32 Privileged Resource Architecture*.

[4] Casio. *Operation Guide, Module 593*.

[5] Maxim Integrated. *DS3231 Extremely Accurate I2C-Integrated RTC/TCXO/Crystal*, datasheet.

[6] STMicroelectronics. *Reference manual RM0008: STM32F10x*.

TODO: Kiểm lại ấn bản/năm từng mục trước khi nộp.